

ANDROID STUDIO

KOTLIN-BILDUMAK: ZERRENDAK ETA MULTZOAK

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Nola sortu zerrenda bat.
2. Nola sortu zerrendak beste zerrenda bateko elementu bat gehituz edo ezabatuz.
3. Bilduma aldaezinen eta aldagarrien arteko aldea.
4. Nola lortu elementu bakar bat zerrenda batetik.
5. Bilduma-eragiketak, hala nola filter, map eta sorted.
6. Zerrenda baten eta multzo baten arteko aldea.

NOLA SORTU ZERRENDÀ BAT

Orain arte, balio indibidualak bezalako aldagaiekin baino ez dugu lan egin. Kotlinen idaztea askoz interesgarriagoa bihurtzen da aldagaiak elkartzen hasten garenean, aldagaiekin bilduma gisa lan egin ahal izateko.

Libby irakurle amorratua da. Liburu berrienbila dabil beti, eta norbaitek liburu on bati buruz hitz egiten dionean, izenburua idazten du paperezko orri batean daraman zerrenda batean. Hauek dira gaur egun zure zerrendan dauden izenburuak:

Books to Read
Tea with Agatha
Mystery on First Avenue
The Ravine of Sorrows
Among the Aliens
The Kingsford Manor Mystery

ZERRENDETARAKO SARREERA

Has gaitezen Kotlineko bilduma mota ohikoenetako batekin: zerrenda bat. Zerrenda bat sortzea erraza da: soilik deitu `listOf()` nahi dituzun balioekin eta bereizi komekin. Libby-ren kodea eguneratu dezagun zerrenda bat erabil dezan.

```
val booksToRead = listOf(  
    "Tea with Agatha",  
    "Mystery on First Avenue",  
    "The Ravine of Sorrows",  
    "Among the Aliens",  
    "The Kingsford Manor Mystery",  
)
```

ZERRENDETARAKO SARREERA

Kode honek Libbyren eskuz idatzitako zerrendaren antz handia du.



ZERRENDETARAKO SARRERA

Eskuz idatzitako zerrendak eta Kotlinen zerrendak asko dute komunean:

1. Lehenik eta behin, biek dute izen bat. Kotlinen, zerrenda duen aldagaiaren izena paperezko zerrendaren izena bezalakoa da.
2. Jarraian, bi zerrendek elementuak dituzte: kasu honetan, liburuen izenburuak. Kotlinen, zerrenda bateko elementuei elementu* esaten zaie.
3. Azkenik, bi zerrendek izenburuak ordena jakin batean dituzte.

Println () ere erabil dezakezu bildumako aldagai batekin.

```
println(booksToRead)
```

BILDUMAK ETA MOTAK

Kotlinen bildumekin lan egiten dugunean, kontuan hartu beharreko bi mota desberdin ditugu.

1. Erabiltzen ari garen bilduma mota.
2. Bildumako elementu motak.

Bi elementu horiek batera bildumaren aldagaiaren mota orokorra zehazten dute. Adibide konkretuarekin jarraituz.

1. Bilduma List bat da.
2. Bildumako elementuen mota String da.

BILDUMAK ETA MOTAK

```
val booksToRead = listOf(  
    "Tea with Agatha",  
    "Mystery on First Avenue",  
    "The Ravine of Sorrows",  
    "Among the Aliens",  
    "The Kingsford Manor Mystery",  
)
```

The kind of collection
we're using is a List.

Each of these elements
is a String.

BILDUMAK ETA MOTAK

Bi gauza horiek jakiten dituzunean, erraz idatzi ahal izango duzu bildumako aldagai baten mota. Lehenik, bilduma-mota jarri behar duzu, eta, ondoren, elementu-mota, ezkerreko kortxete angeluar baten artean. Horrela, **booksToReades List<String>** mota.

Type of the collection

Type of the elements

List<String>

BILDUMAK ETA MOTAK

Zerrenda berridatz dezagun, oraingoan bildumako booksToRead aldakorrerako informazio mota esplizituki sartuta.

```
val booksToRead: List<String> = listOf(  
    "Tea with Agatha",  
    "Mystery on First Avenue",  
    "The Ravine of Sorrows",  
    "Among the Aliens",  
    "The Kingsford Manor Mystery",  
)
```

Mota hau generiko baten instantzia bat da.

ELEMENTU BAT ERANSTEA ETA EZABATZEA

Libbyk bere lagun Rebeccaren beste liburu handi baten berri izan du. Izenburu berri hau gehitzeko prest dago, Beyond the Expanse, zure zerrendan. Nola egin dezakezu?

Jakina, besterik gabe, argumentu bat gehiago gehitu ahal izango dut listOf() amaieran. Baina zer gertatzen da izenburua gehitzen badu zerrenda sortu ondoren?

Kotlinen, zerrenda bat sortzeko **listOf ()** deitu duzunean, zerrenda hori ezin da aldatu. Ezin diozu ezer gehitu, ezta ezer kendu ere. Programazioan "aldatzeko" hitz dotorea **mutate** da; beraz, elementuak gehitzeko edo ezabatzeko aukerarik ematen ez dizun zerrenda bati **zerrenda aldaezina** deitzen zaio.

ELEMENTU BAT ERANSTEA ETA EZABATZEA

```
val booksToRead = listOf(  
    "Tea with Agatha",  
    "Mystery on First Avenue",  
    "The Ravine of Sorrows",  
    "Among the Aliens",  
    "The Kingsford Manor Mystery",  
)  
  
val newBooksToRead = booksToRead + "Beyond the Expanse"
```

ELEMENTU BAT ERANSTEA ETA EZABATZEA

Kode honetan, booksToRead + "Beyond the Expanse" Zerrenda berri gisa ebaluatzen den adierazpena da. Beraz, kode hau exekutatzeko amaitzen denean, bildumako bi aldagai ditugu: booksToRead eta newBooksToRead.

Books to Read	New Books to Read
Tea with Agatha	Tea with Agatha
Mystery on First Avenue	Mystery on First Avenue
The Ravine of Sorrows	The Ravine of Sorrows
Among the Aliens	Among the Aliens
The Kingsford Manor Mystery	The Kingsford Manor Mystery
	Beyond the Expanse

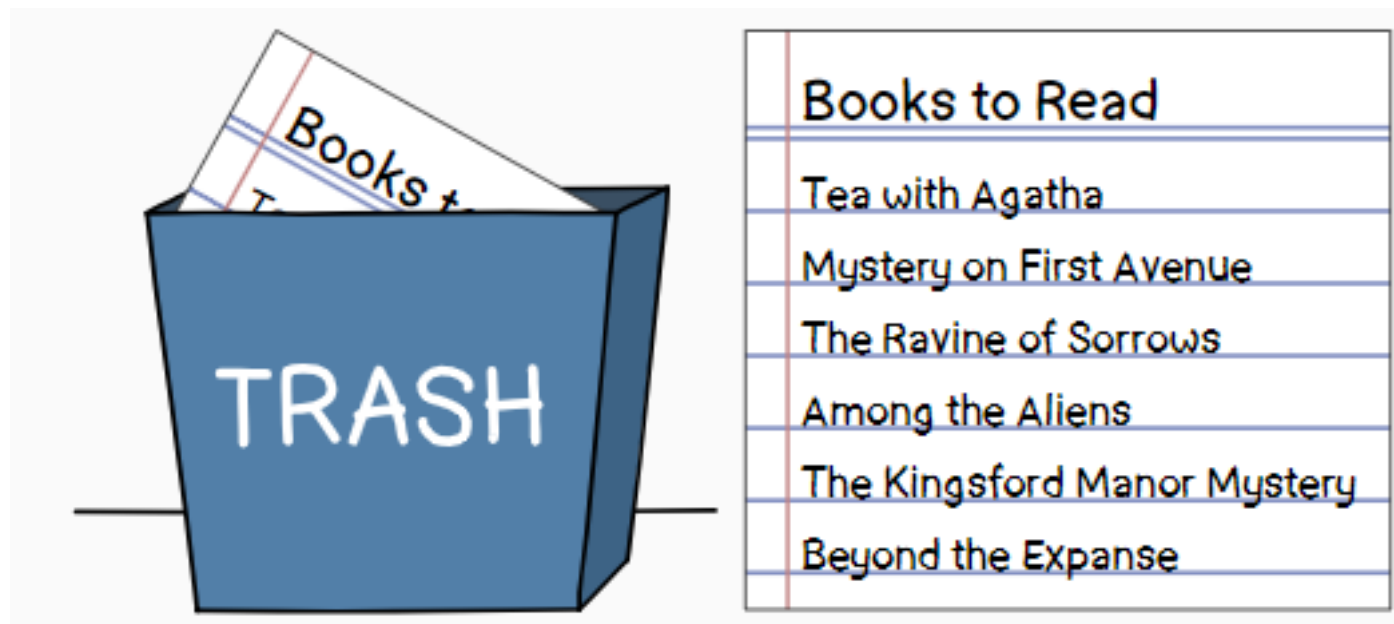
ELEMENTU BAT ERANSTEA ETA EZABATZEA

Aldagaien kapitulua gogoratzen badugu, aldagai bat val edo var bidez adieraz daiteke, eta horrek bilduma bat duten aldagaiak barne hartzen ditu. VAR bidez adierazteak ez du esan nahi elementuak gehitu edo ezabatu ditzakezunik. Hala ere, var-ek aukera ematen dizu beste zerrenda aldaezin bat esleitzeko. Beraz, booksToReadvariable val-etik a var-a aldatzean, zerrenda berria lehendik dagoen aldagaiaren izenari esleitu dakioke, honela:

```
var booksToRead = listOf(  
    "Tea with Agatha",  
    "Mystery on First Avenue",  
    "The Ravine of Sorrows",  
    "Among the Aliens",  
    "The Kingsford Manor Mystery",  
)  
  
booksToRead = booksToRead + "Beyond the Expanse"
```

ELEMENTU BAT ERANSTEA ETA EZABATZEA

Paperezko zerrenda zaharra zakarrontzira bota eta zerrenda berriari aurrekoaren izen bera ematea bezala da.



ELEMENTU BAT ERANSTEA ETA EZABATZEA

Libbyren zerrendak sei izenburu ditu. Zerrenda eguneratu zuela uste zuenean, Rebeccaren berri izan zuen berriro. "Badakizu, Among the Aliens irakurri nuen joan den astean. Ez zen oso ona izan ", esan zuen. "Ez zenuke hori irakurtzen nekatu behar".

Libbyk elementu hori bere zerrendatik kendu nahiko luke. Pentsa dezakezunez, zerrenda bateko elementu bat antzeko moduan ezaba dezakezu, baina operadore gehiago erabili beharrean, operadore gutxiago erabili.

ELEMENTU BAT ERANSTEA ETA EZABATZEA

```
var booksToRead = listOf(  
    "Tea with Agatha",  
    "Mystery on First Avenue",  
    "The Ravine of Sorrows",  
    "Among the Aliens",  
    "The Kingsford Manor Mystery",  
)  
  
booksToRead = booksToRead + "Beyond the Expanse"  
booksToRead = booksToRead - "Among the Aliens"
```

```
booksToRead = booksToRead + "Beyond the Expanse" - "Among the Aliens"
```

ZERRENDAA ETA ZERRENDAA ALDAKORRA

Orain arte, Kotlin List zerrenda normal bat erabiltzen aritu gara, baina ezin da aldaketarik egin, lehen ikusi dugun bezala. Horren ordez, zerrenda berri bat sortu behar izan genuen operadore bat erabiliz.

Hala ere, Kotlinek beste zerrenda mota bat ere eskaintzen du aldatzeko aukera ematen duena. Zerrenda horiek aldaketak egiteko aukera ematen dutenez, zerrenda **mutagarri** deitzen zaie eta **MutableList** mota bat dute.

Zerrenda aldakor bat erabiltzean, **add ()** eta **remove ()** funtzioak erabil ditzakezu elementuak gehitzeko edo ezabatzeke, honela:

ZERRENDΛ ETΛ ZERRENDΛ ALDAKORRA

```
val booksToRead: MutableList<String> = mutableListOf(  
    "Tea with Agatha",  
    "Mystery on First Avenue",  
    "The Ravine of Sorrows",  
    "Among the Aliens",  
    "The Kingsford Manor Mystery",  
)  
  
booksToRead.add("Beyond the Expanse")  
booksToRead.remove("Among the Aliens")
```

ZERREND A ETA ZERREND A ALDAKORRA

List bat erabiltzen ari bazara, erabili + eta - eragileak, eta MutableList bat erabiltzen ari bazara, erabili add () eta remove () funtzioak.

ELEMENTU BAT ZERRENDAA

BATETIK LORTU

Nola lortzen da zerrendako lehen titulua Kotlinen?

First Item



Books to Read
Tea with Agatha
Mystery on First Avenue
The Ravine of Sorrows
The Kingsford Manor Mystery
Beyond the Expanse

ELEMENTU BAT ZERRENDAA

BATETIK LORTU

Lehen esan bezala, zerrenda bateko elementuak ordena jakin batean daude, eta ordena hori oso garrantzitsua da zerrendatik elementu bat ateratzeko. Horrela funtzionatzen du:

Zerrendako elementu bakoitzari zenbaki bat esleitzen zaio, aurkibidea izenekoa, zerrendan duen posizioaren arabera. Lehenengo elementuak 0ko indizea du, bigarrenak 1koa, hirugarrenak 2koa, eta horrela hurrenez hurren.

```
val booksToRead = listOf(  
0 ..... "Tea with Agatha",  
1 ..... "Mystery on First Avenue",  
2 ..... "The Ravine of Sorrows",  
3 ..... "The Kingsford Manor Mystery",  
4 ..... "Beyond the Expanse"  
)
```

ELEMENTU BAT ZERRENDAN BATETIK LORTU

Erraza da zerrenda batetik elementu bat lortzea aurkibidea ezagutzen denean. Besterik gabe, deitu **get ()** kide funtzioa eta pasa aurkibidea argumentu gisa. Adibidez, Libbyk zerrendako lehen elementua lor dezake **get (0)** dei bat egitean, honela:

```
val booksToRead = listOf(  
    "Tea with Agatha",  
    "Mystery on First Avenue",  
    "The Ravine of Sorrows",  
    "The Kingsford Manor Mystery",  
    "Beyond the Expanse"  
)  
  
val firstBook = booksToRead.get(0)  
println(firstBook) // Tea with Agatha
```

ELEMENTU BAT ZERRENDAN BATETIK LORTU

get () funtzio zuzenean deitu ordez, indexatu sarbidea operadorea erabili ahal izango duzu, **[]** kortxete batekin idazten da, aurkibidea erdian. Hurrengo zerrendako kodeak aurreko kodeak egiten duen gauza bera egiten du.

```
val firstBook = booksToRead[0]  
println(firstBook) // Tea with Agatha
```


BUKLEAK(BEGIZTAK) ETA ITERAZIOAK

`println(booksToRead)` erabili eskero ondorengoa lortzen dugu:

```
[Tea with Agatha, Mystery on First Avenue, The Ravine of Sorrows, The Kingsford Manor Mystery,
```

Bertikalean ikusi nahi izanez gero:

```
println(booksToRead[0])  
println(booksToRead[1])  
println(booksToRead[2])  
println(booksToRead[3])  
println(booksToRead[4])
```

BUKLEAK(BEGIZTAK) ETA ITERAZIOAK

Hala ere, horrelako kodea idaztea nahiko aspergarria da. Gainera, erraza izango litzateke errore bat egitea elementuak ordenaz kanpo inprimatzean edo elementu bera halabeharrez behin baino gehiagotan inprimatzean.

Zerrendako elementu bakoitzerako kode bera idatzi beharrean, zer gertatuko litzateke Kotlinek elementu bakoitza banan-banan zeharkatu eta bakoitzari `println()` deitu ahal izango balio?

Zorionez, hau oso erraza da Kotlinen. **ForEach()** funtzioa erabil dezakegu. Horrela gertatzen da

```
booksToRead.forEach { element ->
    println(element)
}
```

BUKLEAK(BEGIZTAK) ETA ITERAZIOAK

Kotlinek kode hori exekutatzen duenean, println (element) beherantz exekutatzen du lehen elementurako, gero berriro igotzen da eta berriro beherantz exekutatzen du bigarren elementurako, gero berriro igotzen da eta berriro beherantz exekutatzen du hirugarren elementurako, eta horrela hurrenez hurren. Kode-lerro hori behin eta berriz ibiltzean, zirkuluan bueltaka ariko balitz bezala da, hau bezala:

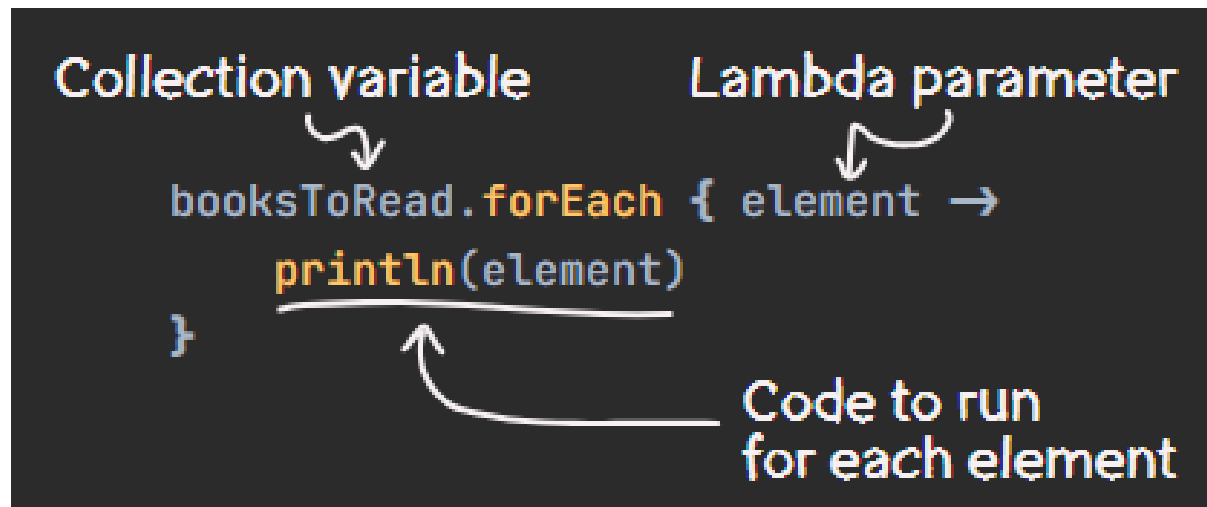


```
println(element)
```

The diagram shows the Kotlin code `println(element)` in orange and white text on a dark background. Below the code, there are four large, stylized blue loops that represent the iterative nature of the loop. Above the loops, five lines of text are written in a yellow, slightly tilted font, representing the sequence of elements being printed: "Tea with Agatha", "Mystery on First Avenue", "The Ravine of Sorrows", "The Kingsford Manor Mystery", and "Beyond the Expanse".

BUKLEAK(BEGIZTAK) ETA ITERAZIOAK

forEach () bildumaren aldagaietan dagoen funtzio kide bat da. Lambda bat onartzen duen goragoko mailako funtzio bat da. Lambda hori da Kotlinek bildumako elementu "bakoitzerako" exekutatzeari nahi duzun kodea.



```
Collection variable      Lambda parameter
    ↘                   ↘
booksToRead.forEach { element →
    println(element)
}
```

Code to run
for each element

The diagram illustrates the syntax of the `forEach` function in Kotlin. It shows a code snippet: `booksToRead.forEach { element → println(element) }`. Hand-drawn arrows point from the labels 'Collection variable' and 'Lambda parameter' to `booksToRead` and `element` respectively. Another arrow points from the label 'Code to run for each element' to the lambda body `println(element)`.

BUKLEAK(BEGIZTAK) ETA ITERAZIOAK

Hemen element parametroa izendatu dugu, baina title izenda zenezakeen haren ordeez. Alternatiba gisa, lambda horrek parametro bakarra duenez, 'it' parametro inplizitua erabil dezakezu haren ordeez, eta horrek atsegina eta zehatza egiten du. Izan ere, dena lerro batean jar dezakegu:

```
booksToRead.forEach { println(it) }
```

BUKLEAK(BEGIZTAK) ETA ITERAZIOAK

Emaitza, bietan, Libbyk nahi zuena da: liburuen izenburuak bertikalki inprimatzen dira, paperezko ohar-blokean bezala!

Tea with Agatha
Mystery on First Avenue
The Ravine of Sorrows
The Kingsford Manor Mystery
Beyond the Expanse

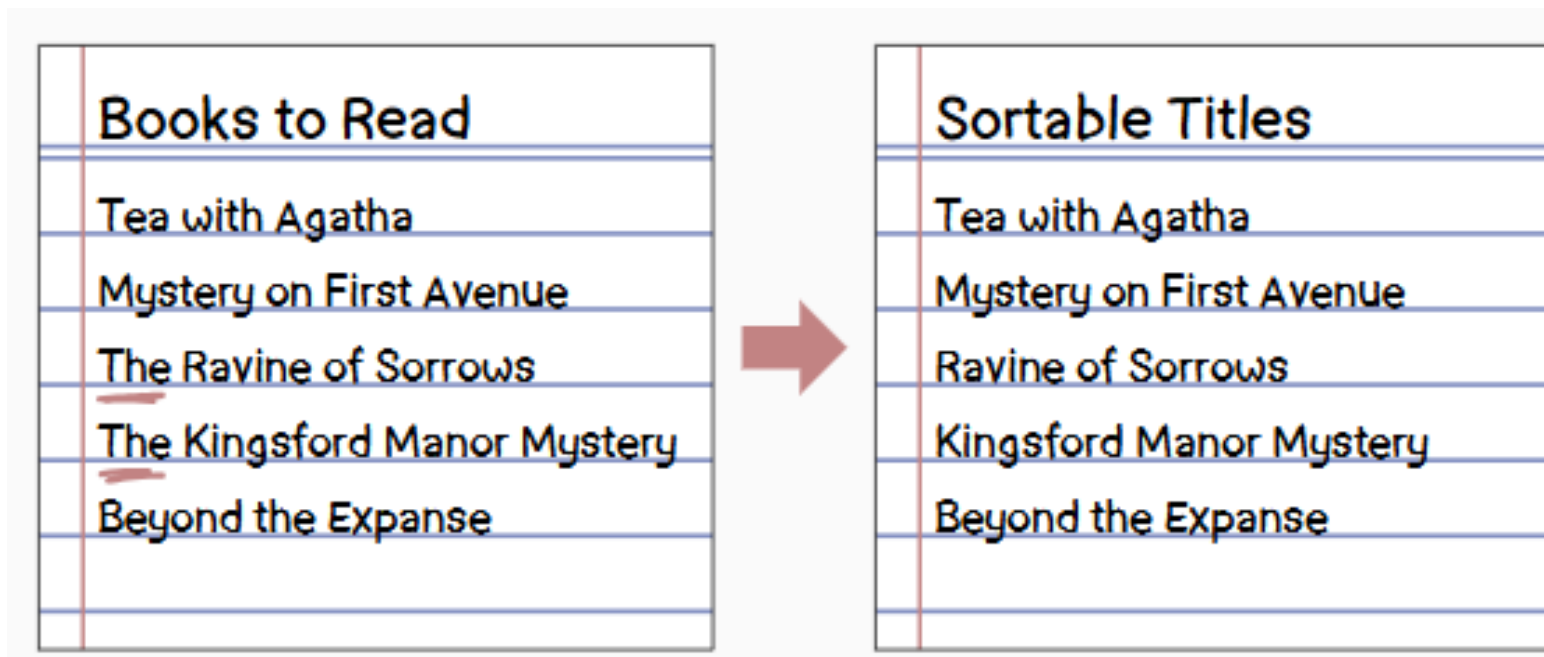
KOBRANTZA-ERAGIKETAK

Libby prest dago bere liburu zerrenda irakurtzeko interesa duten beste pertsona batzuekin partekatzeko, Nolan lagunarekin hasita. Hala ere, harentzat zerrendaren kopia bat egiten duenean, titulu batzuetan aldaketak egin nahi ditu.

"Asko gustatuko litzaidake 'the' hitza izenburu bakoitzaren hasieratik kentzea", pentsatu zuen Libbyk. "Horrela, alfabetikoki ordenatu ahal izango ditut, eta 'The' -ekin hasten diren izenburuak ez dira pilatuko".

BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

Batzuetan, lehendik dagoen bilduma batetik abiatuta bilduma berri bat sortzen duzunean, elementuetako bat edo gehiago nolabait bihurtu nahi izaten dituzu. Libbyren kasuan, "The" hitza ezabatu nahi du izenburu baten hasieran agertzen denean,



BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

Bihurketa hori izenburu guztietan egin aurretik, has gaitezen horietako bakar batekin. StringObjektuek `removePrefix` bat dute (), katearen hasierako hitzak ezabatzeke erabil dezakezun funtzioa. Hemen erakutsiko dizugu nola erabil dezakezun:

```
val sortableTitle = "The Kingsford Manor Mystery".removePrefix("The ")  
println(sortableTitle) // Kingsford Manor Mystery
```

BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

Primeran! Orain `removePrefix()` funtzio hau zerrendako elementu bakoitzari aplikatzea besterik ez da falta!

"Agian `forEach()` erabil dezaket, badakit zerrendako elementu bakoitzaren gainean lan egiten duela", pentsatu zuen Libbyk. Damutu egin zen eta kode hau idatzi zuen:

```
val sortableTitles: MutableList = mutableListOf()

booksToRead.forEach { title ->
    sortableTitles.add(title.removePrefix("The "))
}

sortableTitles.forEach { println(it) }
```

BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

“Beno, horrek funtzionatzen du”, pentsatu zuen Libbyk. “Baina pixka bat konplikatua da eta kode asko idatzi behar da...”

Arrazoia hori zaila da Libby bilduma berri bat sortu nahi zuen, baina `forEach()` ez du. Dagoen bilduma batean lambda exekutatu eta gero Unit itzultzen du. Benetan behar duena lambda egiten duen eta lambda horren emaitza bilduma berri bateko elementu gisa jasotzen duen bilduma-eragiketa bat da.

BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

Kotlinen, bilketa eragiketa horri `map ()` deitzen zaio. Ondoren, Libby erabili ahal izango duzu "The" hitza izenburuak hasieran kentzeko bilduma berrian erakusten da:

```
val sortableTitles = booksToRead.map { title ->
    title.removePrefix("The ")
}
```

BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

Kode honek aurreko zerrendaren berdina egiten du (emaitza Listen MutableList baten leku aldaezina denean izan ezik). `ForEach ()` bezala, `map ()` funtzioak lambda deitzen du behin zerrendako elementu bakoitzarekin. Hala ere, `forEach ()` ez bezala, `map ()` lambda emaitza erabiliko du iterazio bakoitzean zerrenda berri bat sortzeko.

BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

```
title.removePrefix("The ")
```

"Tea with Agatha"
"Mystery on First Avenue"
"The Ravine of Sorrows"
"The Kingsford Manor Mystery"
"Beyond the Expanse"

"Tea with Agatha"
"Mystery on First Avenue"
"Ravine of Sorrows"
"Kingsford Manor Mystery"
"Beyond the Expanse"

BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

Zerrendako elementu bakoitza inprimatzean, The Ravine of Sorrows eta The Kingsford Manor Mystery eguneratu egin direla ikus daiteke, "The" hitza hasieran egon ez dadin.

Tea with Agatha
Mystery on First Avenue
Ravine of Sorrows
Kingsford Manor Mystery
Beyond the Expanse

BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

Ikus dezagun map () funtzioa hurbilagotik:

The diagram illustrates the Kotlin `map` function with the following code and annotations:

```
val sortableTitles = booksToRead.map { title →  
    title.removePrefix("The ")  
}
```

Annotations:

- New list variable**: Points to `sortableTitles`.
- Original list variable**: Points to `booksToRead`.
- Lambda parameter**: Points to `title`.
- Evaluates to a value that becomes an element in the new list.**: Points to the lambda body `title.removePrefix("The ")`.

BILDUMEN MAPAKETA: ELEMENTUEN BIHURKETA

- `forEach`-en antzekoa `()`, `map` `()` funtzioa lambda bat hartzen duen ordena goreneko funtzio bat da.
- Lambda hori behin exekutatuko da zerrendako elementu bakoitzerako.
- Lambdaren emaitza bilduma berriko elementu bat izango da.
- `Map` `()` funtzioak bilduma berri hori itzultzen du.

`forEach` `()` eta `map` `()` funtzioak bildumako eragiketak deitzen dira, eragiketaren bat bilduma batean (hau da, zerbait egiten dute) egiten duten funtzioak direlako.

BILDUMAK ORDENATZEA

ForEach () eta map () funtzioak Kotlinen bilketa eragiketa ugarietako bi baino ez dira. Nahiko erabilgarria izan daitekeen beste bati sorted () esaten zaio.

Map () funtzioak bilduma bat itzultzen duenez, Libbyk sorted () dei dezake map () deitu eta berehala, honela:

```
val sortedTitles = booksToRead.map { title -> title.removePrefix("The ") }.sorted()
```

BILDUMAK ORDENATZEA

Zozketako elementuak inprimatu zituenean, espero zuen emaitza ikusi zuen!

Beyond the Expanse
Kingsford Manor Mystery
Mystery on First Avenue
Ravine of Sorrows
Tea with Agatha

BILDUMAK ORDENATZEA

Irakurketa errazteko, bilketa-eragiketa bakoitza bere lerroan joan daiteke, honela:

```
val sortedTitles = booksToRead  
    .map { title -> title.removePrefix("The ") }  
    .sorted()
```

BILDUMAK IRAGAZTEA (FLITRA): ELEMENTUAK SARTZEA ETA EZ AIPATZEA

Libby hunkituta dago! Orain liburu zerrenda bat du, alfabetikoki ordenatuta, Nolanekin partekatzeko. "Zure liburu zerrenda ikusteko irrikitan nago", esan zuen Nolanek. "Gogoratu: misteriozko eleberriak bakarrik irakurtzen ditut!"

"Misterioak bakarrik...?" errepikatu zuen Libbyk. "Ondo da", pentsatu zuen bere artean, "misterioak ez diren tituluak ezabatu behar ditut azkenekoz". Orri bat atera zuen bere ohar-bloketik eta zerrenda pertsonalizatu bat idatzi zuen Nolanentzat bakarrik, misterioa ez den edozein izenburu alde batera utziz.

BILDUMAK IRAGAZTEA (FLITRA): ELEMENTUAK SARTZEA ETA EZ AIPATZEA

Books to Read

Beyond the Expanse

Kingsford Manor Mystery

Mystery on First Avenue

Ravine of Sorrows

Tea with Agatha



Books for Nolan

Kingsford Manor Mystery

Mystery on First Avenue

BILDUMAK IRAGAZTEA (FLITRA): ELEMENTUAK SARTZEA ETA EZ AIPATZEA

"Nola egin dezaket hau Kotlinen?" galdetu zion bere buruari. Ziurrenik asmatu duzun bezala, Kotlinek bilketa eragiketa bat du, eta horrek erraza egiten du, eta, espero bezala, `filter()` deitzen da.

Hautsak eta nahi ez diren alergenok aire girotuaren sistemara pasatzea eragozten duen aire-iragazki baten antzera, Kotlinen zerrenda-iragazki batek blokeatu egiten ditu zerrenda berri batera pasatzea nahi ez duten elementuak.

BILDUMAK IRAGAZTEA (FILTER): ELEMENTUAK SARTZEA ETA EZ AIPATZEA

Erabil dezagun `filter()` funtzioa liburuen zerrenda izenburuan "Misterioa" dutenei bakarrik iragazteko:

```
val booksForNolan = booksToRead
    .map { title -> title.removePrefix("The ") }
    .sorted()
    .filter { title -> title.contains("Mystery") }
```


BILDUMAK IRAGAZTEA (FLITRAR): ELEMENTUAK SARTZEA ETA EZ AIPATZEA

`Filter ()` funtzioa aurreko `map ()` funtzioaren antzekoa da: lambda bat argumentu gisa hartzen du eta lambda hori jatorrizko zerrendako izenburu bakoitzeko behin deituko da. `map ()` Hala ere, funtzioa ez bezala, lambda for `filter ()` funtzioak Boolean bat itzuli behar du. Elementu bat trukutzen badu, orduan elementu hori bilduma berrira pasatuko da (hau da, kasu honetan `booksForNolanen`). Huts egiten badu, bilduma berria alde batera utziko du.

BILDUMAK IRAGAZTEA (FLITRAR): ELEMENTUAK SARTZEA ETA EZ AIPATZEA

"Tea with Agatha"
"Mystery on First Avenue"
"The Ravine of Sorrows"
"The Kingsford Manor Mystery"
"Beyond the Expanse"

`title.contains("Mystery")`

false true false true false

BILDUMAK IRAGAZTEA (FILTER): ELEMENTUAK SARTZEA ETA EZ AIPATZEA

Jarraian, `filter()` funtzioa nola erabili azaltzen da banakatuta:

```
val booksForNolan = booksToRead.filter { title →  
    title.contains("Mystery")  
}
```

Determines whether this element
is included in the new list.

BILDUMAK IRAGAZTEA (FLITRA): ELEMENTUAK SARTZEA ETA EZ AIPATZEA

Zerrendako elementu bakoitza inprimatzean, hauxe ikusi zuen Libbyk:

Kingsford Manor Mystery
Mystery on First Avenue

BILTZEKO ERAGIKETA-KATEAK

Kotlinen, ohikoa da hainbat bilketa-eragiketa elkarrekin jartzea, modu horretan, bata bestearen atzetik. Hori egiten dugunean, bilketa-eragiketak kateatzea esaten zaio: eragiketa bakoitza katearen kate-maila bat bezalakoa da. Kode-zerrenda honetan, `sorted()` eta `filter()` izeneko `map()` -ak kateatuta daude.

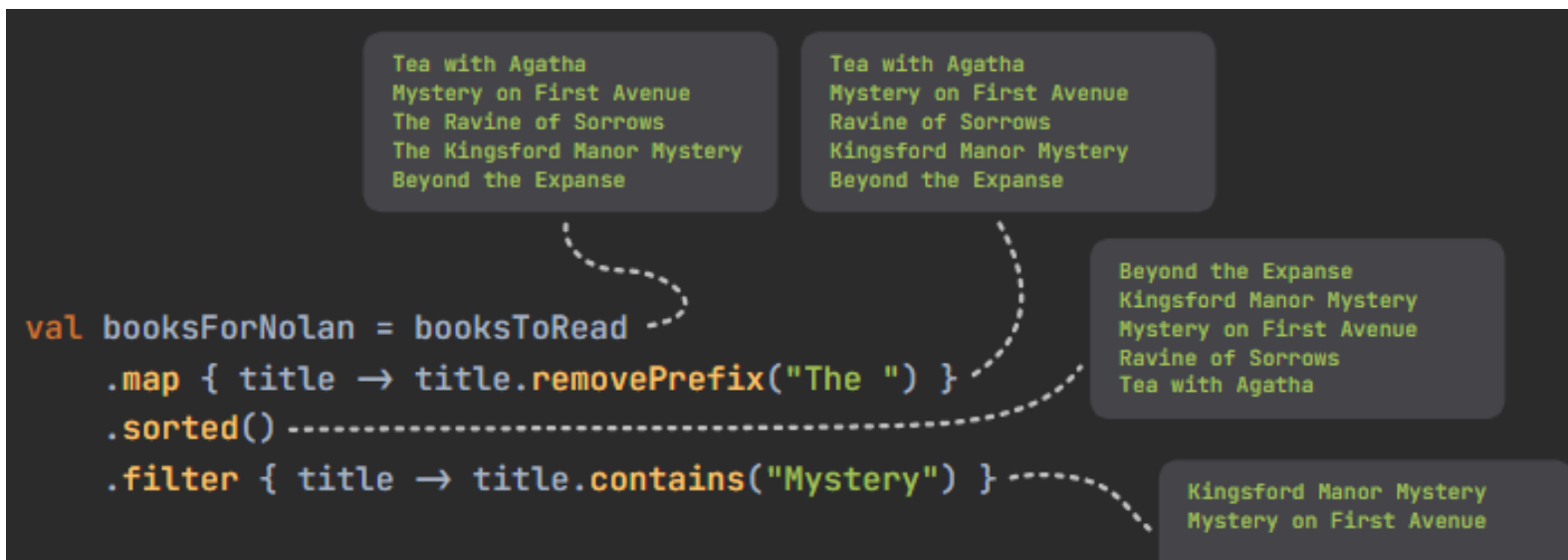
```
Collection operation chain {  
    val booksForNolan = booksToRead  
    .map { title → title.removePrefix("The ") }  
    .sorted()  
    .filter { title → title.contains("Mystery") }  
}
```

BILTZEKO ERAGIKETA-KATEAK

Kontuan izan eragiketa-katea ez dela zerrenda bakarra aldatzen ari. Izan ere, eragiketa horietako bakoitzak zerrenda berri bat sortzen du. Finalfilter () eragiketaren bidez sortzen den zerrenda booksForNolan aldagaiari esleitzen zaion zerrenda da. Tarteko zerrendak (hau da, katearen barruko bilketa-eragiketen bidez sortzen diren zerrendak) katearen hurrengo eragiketan erabiltzen dira, baina ez dira aldagai bati ere esleitzen.

BILTZEKO ERAGIKETA-KATEAK

Hala ere, garrantzitsua da oraindik ere tarteko zerrenda horiek kontuan hartzea. Hurrengo irudiak katearen urrats bakoitzean esku hartzen duen zerrenda erakusten du.



BESTE KOBRANTZA-ERAGIKETA BATZUK

Kotlinek erabiltzen errazak diren beste bilketa-eragiketa asko ditu. Ideia bat emateko bakarrik, hemen beste batzuk baliagarriak izan dakizkizuke.

- **drop (3)** - Zerrenda berriak ez ditu jatorrizko zerrendako lehen 3 elementuak aipatzen.
- **take (5)** - Zerrenda berriak jatorrizko zerrendako lehen 5 elementuak bakarrik erabiltzen ditu.



BESTE KOBRANTZA-ERAGIKETA BATZUK

- **distinct ()** - Zerrenda berriak bikoiztutako elementuak alde batera utziko ditu, elementu bakoitza behin bakarrik sar dadin.
- **Reversed ()** - Zerrenda berriak jatorrizkoaren elementu berak izango ditu, baina bere ordena alderantzizkoa izango da.

MULTZOETARAKO SARRERA

Kapitulu hau amaitu aurretik, merezi du aipatzea zerrendak ez direla Kotlinen bilduma mota bakarra. Zerrendak dira ziurrenik erabilienak, baina beste bilduma erabilgarri mota bat multzo deitzen da. Zerrendak baliagarriak dira beren elementuak ordena jakin batean daudela bermatzeko, baina multzoak baliagarriak dira elementu bakoitza beti bakarra dela bermatzeko.

Adibidez, Nolanen misterio gogokoenaren egileak, Slim Chanceryk, hiru liburu idatzi ditu, eta Nolan harro dago multzo osoa duela esateaz.

MULTZOETARAKO SARRERA

Kotlinen multzo bat sortzea zerrenda bat sortzea bezain erraza da. Soilik erabili `setOf ()` edo `mutableSetOf ()`, `listOf ()` edo `mutableListOf ()` erabili beharrean.

```
val booksBySlim: Set = setOf(  
    "The Malt Shop Caper",  
    "Who is Mrs. W?",  
    "At Midnight or Later",  
)
```

MULTZOETARAKO SARRERA

Elementu bat jada balio hori duen multzo bati gehitzen dionean, multzoa aldaketarik gabe geratuko da.

```
val booksBySlim: MutableSet = mutableSetOf(  
    "The Malt Shop Caper",  
    "Who is Mrs. W?",  
    "At Midnight or Later",  
)  
  
booksBySlim.add("The Malt Shop Caper")  
  
println(booksBySlim)  
// [The Malt Shop Caper, Who is Mrs. W?, At Midnight or Later]
```

MULTZOETARAKO SARRERA

Kontuan izan multzo batek ez duela bere elementuen ordena bermatzen multzo horretan bilketa-eragiketa bat inprimatzen edo erabiltzen duenean. Baliteke elementuak gehitu zituen ordena berean egotea, baina ez dago horren mende!

Multzoek ez dutenez ordena berezirik beren elementuetan, elementuek ez dute indizirik. Horregatik, taldeek ez dute `get ()` funtzio bat ere sartzen!

MULTZOETARAKO SARRERA

Hona hemen funtsezko ondorioa:

1. Zerrendek elementuak dituzte, ordena bermatuan, eta kopiak izan ditzakete.
2. Multzoek ordena berezirik gabeko elementuak dituzte, eta bermatzen da ez dutela bikoizturik.

Zerrenda bat multzo bihur dezakezu, edo alderantziz. `ToSet ()` edo `toList ()` besterik ez du erabiltzen. Gogoan izan zerrenda bat multzo bihurtzen baduzu, bikoiztutako elementuak galduko dituzula eta ordena desberdina izan daitekeela.

MULTZOETARAKO SARRERA

```
val bookList = listOf(  
    "The Malt Shop Caper",  
    "At Midnight or Later",  
    "The Malt Shop Caper",  
)  
  
val bookSet = bookList.toSet()           // bookSet has two elements  
val anotherBookList = bookSet.toList()  // anotherBookList also has two elements
```